

ETF Tracker

模組拆分規劃說明書

版本：v1.0 日期：2026-06-30

一、專案現況

目前 ETF Tracker 後端邏輯集中於單一 app.py (約 3,000 行)，涵蓋資料庫、爬蟲、認證、持倉、AI題材股等多個功能領域。隨著功能持續擴增，建議依職責邊界拆分為獨立模組，以提升可維護性與可測試性，並支援 Subagent 平行開發。

二、建議目錄結構

```
etf-tracker/
├── app.py # Flask init + Blueprint 註冊 + 啟動 (精簡至 <100 行)
├── db.py # DB 連線 + init_db() + 資料表定義
├── scrapers/
│   ├── holdings.py # ETF 持股爬蟲 (MoneyDJ / etfinfo / pocket.tw)
│   ├── prices.py # 股價爬蟲 (Yahoo Finance V8 / TWSE)
│   └── changes.py # 每日操作日報爬蟲 (etfinfo.tw/active)
├── routes/
│   ├── auth.py # Flask Blueprint: /api/auth/*
│   ├── etf.py # Flask Blueprint: /api/etf/*、/api/prices
│   ├── portfolio.py # Flask Blueprint: /api/portfolio
│   ├── admin.py # Flask Blueprint: /api/admin/*、/admin
│   └── ai_stocks.py # Flask Blueprint: /api/ai-stocks、/ai-stocks
├── workers/
│   └── background.py # 所有背景執行緒 (warmup / live_price / prev_close / scheduler)
└── static/ # HTML 前端頁面 (index / login / admin / ai_stocks)
```

三、Subagent 模組分類總覽

以下 5 個 Subagent 可同步平行作業，相依性低，開發期間不會互相衝突。

Subagent	負責模組	主要職責	核心功能摘要	相依性
db-agent	db.py	資料庫連線管理	get_db()、init_db()、 8 個資料表定義 (users/portfolios/themes/ etf_configs/etf_changes_h istory/ stock_close_cache/ ai_stock_picks/ai_stock_d ata)	完全獨立 (無外部依賴)
scraper-agent	scrapers/ holdings.py prices.py changes.py	外部資料爬取	持股：MoneyDJ → etfinfo.tw → pocket.tw → 靜態備援 股價：Yahoo Finance V8 → TWSE STOCK_DAY 日報：etfinfo.tw/active	依賴 db.py (股價/日報快取 寫入 DB)
auth-agent	routes/auth.py	用戶認證系統	登入 / 登出 / 註冊 修改密碼 / 重設密碼 bcrypt 密碼雜湊 Flask session 30天	依賴 db.py (users 表)
ai-stocks-agent	routes/ai_stocks .py workers/background nd.py (ai_weekly 部分)	AI 題材股調查	每週精選 10 檔 AI 股 $ROE \times 0.4 + EPS \text{成長} \times 0.4$ + 營收成長 $\times 0.2$ 評分 Yahoo quoteSummary 基本面 週五 18:00 自動排程	依賴 db.py + scrapers/pric es.py
frontend-agent	static/ index.html login.html admin.html ai_stocks.html	前端介面	持股主頁 (Chart.js 圓餅圖) 登入頁 管理後台 (ETF 新增/刪除) AI 題材股調查頁 主題色客製化面板	完全獨立 (純 HTML/JS， 無後端依賴)

* 表格中的「相依性」代表模組間的單向依賴，不代表需要循序開發；各 Subagent 可同時啟動。

四、各模組詳細說明

4.1 db-agent / db.py

職責	集中所有資料庫相關邏輯，是其他所有模組的底層依賴
主要函式	get_db(): 建立 pg8000 連線 (含 SSL) init_db(): 建立 8 個資料表 + 預設帳號

資料表清單	users (帳號)、portfolios (持倉)、themes (UI主題) etf_configs (ETF設定)、etf_changes_history (日報快照) stock_close_cache (收盤價快取) ai_stock_picks (每週精選)、ai_stock_data (基本面快取)
環境變數	DATABASE_URL (Supabase PostgreSQL 連線字串)

4.2 scraper-agent / scrapers/

職責	所有外部資料爬取邏輯，含多層備援與快取寫入
holdings.py	scrape_moneydj_holdings(): MoneyDJ Basic0007B (首選) scrape_etfinfo(): etfinfo.tw NUXT_DATA 解析 (二選) scrape_fhtrust()、scrape_capital(): 投信官網備援
prices.py	scrape_moneydj_price(): Yahoo Finance V8 API (盤中) TWSE STOCK_DAY: 收盤備援 get_stock_close(): 個股收盤價 (含 DB 快取)
changes.py	scrape_daily_changes(): etfinfo.tw/active 操作日報 save_changes_snapshot(): 寫入 etf_changes_history
快取 TTL	持股 300 秒; 股價盤中 15 秒 / 盤後 600 秒

4.3 auth-agent / routes/auth.py

職責	用戶認證全流程，使用 Flask Blueprint 掛載 /api/auth/*
API 端點	POST /api/auth/login 登入 (bcrypt 驗證) POST /api/auth/register 註冊 (密碼最少6字元) POST /api/auth/logout 登出 (清除 session) GET /api/auth/me 取得當前登入用戶 POST /api/auth/change_password 修改密碼 POST /api/auth/reset_password 管理員重設密碼
Session 設定	PERMANENT_SESSION_LIFETIME = 86400 × 30 (30天) SESSION_COOKIE_HTTPONLY = True
環境變數	SECRET_KEY (固定 session 金鑰)

4.4 ai-stocks-agent / routes/ai_stocks.py + workers/

職責	AI 題材股每週精選調查，含基本面資料抓取與排程更新
候選清單	15 檔 AI 概念股 (台積電、鴻海、緯穎、廣達、台達電...等)
評分公式	$ROE \times 0.4 + EPS_{\text{成長}} \times 0.4 + \text{營收成長} \times 0.2$ ，取前 10 名

資料來源	Yahoo Finance V8：即時股價 Yahoo quoteSummary v10：EPS、PE、ROE、淨利率 winvest.tw Proxy：季 EPS 資料
API 端點	GET /api/ai-stocks 取得本週精選（快取1小時） POST /api/ai-stocks/refresh 手動觸發更新（需登入） GET /api/winvest-eps/ EPS Proxy
排程	每週五 18:00 自動執行 refresh_ai_stock_picks()

4.5 frontend-agent / static/

職責	所有前端 HTML/CSS/JS，與後端解耦，可獨立開發測試
index.html	持股主頁：ETF選單側欄、Chart.js 持股圓餅圖 持倉記錄（成本/損益計算）、每日操作日報 主題色客製化面板（6色×預設方案）
login.html	登入 / 註冊 / 重設密碼 三合一頁面
admin.html	ETF 管理：新增（自動偵測來源）/ 刪除 ETF 代號查詢（etfinfo.tw 自動帶名稱） Render.com API 自動更新環境變數
ai_stocks.html	AI 題材股列表（卡片式 + 篩選 Screener） 按標籤篩選、基本面指標顯示

五、模組相依關係

各模組之間的依賴方向為單向（下層不依賴上層），確保模組可獨立測試與部署。

frontend-agent	（完全獨立，純 HTML/JS）	
auth-agent	依賴 →	db.py
scraper-agent	依賴 →	db.py
ai-stocks-agent	依賴 →	db.py + scrapers/prices.py
db-agent	（底層，無依賴）	

六、Subagent 平行作業建議

拆分完成後，以下任務組合可同步平行執行，大幅縮短開發週期：

第一波 (同時啟動)	db-agent : 建立 db.py 並完善型別注解與錯誤處理 frontend-agent : 整理 HTML 至 static/ 目錄, 建立共用 CSS 元件
第二波 (db.py 完成後)	scraper-agent : 抽取 scrapers/ 三支模組, 補充 retry 邏輯 auth-agent : 建立 routes/auth.py Blueprint, 撰寫單元測試
第三波 (scrapers 完成後)	ai-stocks-agent : 完成 ai_stocks.py Blueprint + workers/ 週排程 (同時) frontend-agent 接續 AI 題材股前端 UI 優化